

Machine Learning e Pattern Recognition – Exam Collection

Luca & Mellow

6 giugno 2026

Indice

1	PCA	2
2	LDA	3
3	PCA vs LDA	5
4	Generative Gaussian Models	8
5	LDA vs Tied MVG	10
6	Logistic Regression	12
7	Logistic Regression vs Generative Gaussian Models	15
8	Logistic Regression vs LDA	16
9	Support Vector Machines	18
10	Logistic Regression vs SVM	22
11	Generative Gaussian Models vs SVM	23
12	Gaussian Mixture Models	25
13	Gaussian Mixture Models vs Generative Gaussian Models .	29

1 PCA

Motivation. Dimensionality reduction maps data from an n -dimensional space to an m -dimensional one, with $m \ll n$. The practical purposes are four-fold: **to compress information, remove unwanted variability (noise), simplify classification by reducing the number of parameters to estimate, and visualize the data.**

PCA is the standard linear and unsupervised technique for this task. A sample is projected to the low-dimensional space as $\mathbf{y} = P^T \mathbf{x}$ and reconstructed in the original space as $\hat{\mathbf{x}} = P\mathbf{y} = PP^T \mathbf{x}$. The matrix $P \in \mathbb{R}^{n \times m}$ with orthonormal columns minimizes the average reconstruction error:

$$P^* = \arg \min_P \frac{1}{K} \sum_i \|\mathbf{x}_i - PP^T \mathbf{x}_i\|^2 \quad \text{s.t. } P^T P = I$$

Equivalently, it maximizes the projected variance:

$$\tilde{L}(P) = \text{Tr} \left(P^T \underbrace{\frac{1}{K} \sum_i \mathbf{x}_i \mathbf{x}_i^T}_C P \right)$$

The solution consists of the m eigenvectors of C with the largest eigenvalues. From the decomposition $C = U \Sigma U^T$:

$$P^* = [\mathbf{u}_1, \dots, \mathbf{u}_m]$$

The result is a coordinate system where each axis captures the maximum remaining variance, and the variance along each axis corresponds to the associated eigenvalue.

Non-centered data. If the data are not zero-mean, the first eigenvector points towards the dataset mean — not informative. The mean $\bar{\mathbf{x}}$ should always be subtracted, and C becomes the empirical covariance matrix:

$$C = \frac{1}{K} \sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T$$

Choice of m . m is a hyperparameter: it does not emerge from the PCA solution itself. Two approaches: cross-validation, or a criterion on explained variance — choose the smallest m such that:

$$\frac{\sum_{i=1}^m \sigma_i}{\sum_{i=1}^n \sigma_i} \geq t \quad (\text{e.g. } t = 0.95)$$

The choice of m introduces a trade-off. If m is too large, too many dimensions remain, increasing the number of parameters to estimate in the downstream classifier and the risk of overfitting. If m is too small, discriminant directions may be lost, causing underfitting. Cross-validation allows balancing this trade-off for the specific task.

Limitations. Computing C is expensive in high-dimensional spaces. Most importantly: being unsupervised, PCA does not guarantee to preserve discriminant directions — directions of highest variance do not necessarily coincide with those useful for class separation. **A further practical advantage is that PCA automatically eliminates directions with zero or very low variance, which could cause numerical instability in some classifiers** (e.g., division by zero variance).

2 LDA

Motivation. LDA is a linear **supervised** technique for dimensionality reduction and classification. **Unlike PCA, which maximizes variance without knowledge of class labels, LDA exploits class information to find directions that explicitly separate groups.** The goal is to find the direction $\mathbf{w}$ that maximizes the separation between classes relative to the within-class scatter:

$$\max_{\mathbf{w}} L(\mathbf{w}) = \frac{s_B}{s_W} = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$$

where the scatter matrices are:

$$S_B = \frac{1}{N} \sum_{c=1}^K n_c (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T, \quad S_W = \frac{1}{N} \sum_{c=1}^K \sum_{i=1}^{n_c} (\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)(\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)^T$$

S_B measures how far the class means are from the global mean; S_W measures how spread out the samples are around their class mean. Maximizing the ratio means finding a projection where classes are compact internally and far apart from each other.

Solution derivation. Setting $\nabla_{\mathbf{w}} L = 0$, dividing by the scalar $(\mathbf{w}^T S_W \mathbf{w})$ and multiplying by S_W^{-1} , leads to the eigenvalue problem:

$$S_W^{-1} S_B \mathbf{w} = \lambda(\mathbf{w}) \mathbf{w}, \quad \lambda(\mathbf{w}) = L(\mathbf{w})$$

Maximizing L is thus equivalent to maximizing λ : the optimal direction is the eigenvector of $S_W^{-1} S_B$ with the largest eigenvalue.

Binary case. With two classes, $S_B = k(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T$, which has rank 1, and the solution simplifies in closed form:

$$\mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$$

This formula has a direct interpretation: compute the difference between the two class means and "correct" it for within-class scatter via S_W^{-1} . If the classes have the same within-class scatter ($S_W = I$), the optimal direction is simply the one joining the two means.

The classification rule assigns $\mathbf{x}_t$ to the closest class in the projected space, with threshold $t = \frac{m_1 + m_2}{2}$:

$$C(\mathbf{x}_t) = \begin{cases} C_1 & \mathbf{w}^T \mathbf{x}_t < t \\ C_2 & \mathbf{w}^T \mathbf{x}_t \geq t \end{cases}$$

The threshold t is the mean of the projected class means: it is the optimal choice under the assumption of equal within-class variance and equal prior. If the priors differ, the threshold must be shifted accordingly.

Extension to m dimensions. The m most discriminant directions are collected in $W \in \mathbb{R}^{n \times m}$. The criterion becomes:

$$\max_W \text{Tr}(\tilde{S}_W^{-1} \tilde{S}_B), \quad \tilde{S}_B = W^T S_B W, \quad \tilde{S}_W = W^T S_W W$$

The solution consists of the m eigenvectors of $S_W^{-1}S_B$ with the largest eigenvalues. **An important structural constraint: the rank of S_B is at most $K - 1$, because it is defined by K means constrained to sum to the global mean. As a result, the maximum number of discriminant directions that can be extracted is $K - 1$, regardless of the original data dimension. For binary classification, there is always only one direction.**

Limitations and practical considerations. LDA assumes that the within-class distributions are Gaussian with the same covariance matrix — an assumption often violated in practice. S_W can become singular when the number of samples is less than the data dimension, making S_W^{-1} not computable. **A common solution is to apply PCA as a pre-processing step before LDA: PCA reduces dimensionality by eliminating directions with zero or near-zero variance, making S_W invertible and reducing the risk of overfitting. However, care must be taken not to eliminate discriminant directions with PCA before LDA can find them — the number of PCA components should be chosen carefully, typically via cross-validation.**

3 PCA vs LDA

Both **PCA** and **LDA** are linear dimensionality reduction techniques that map data from an n -dimensional feature space to a lower m -dimensional space ($m \ll n$), but they differ fundamentally in supervision, objective, and the nature of the directions they find.

Goals and Formulation

PCA is an *unsupervised* technique. Its goal is to find an orthogonal projection matrix $P \in \mathbb{R}^{n \times m}$ that best preserves the structure of the data, without any knowledge of class labels. Data is first centered by subtracting the mean $\bar{\mathbf{x}}$, and then PCA seeks directions that capture the most variance. The projection of a sample is $\mathbf{y} = P^T \mathbf{x}$, and the reconstruction in the original space is $\hat{\mathbf{x}} = P\mathbf{y}$.

LDA is a *supervised* technique. Its goal is to find a projection matrix $W \in \mathbb{R}^{n \times m}$ such that the projected data has maximum separation between classes

and minimum spread within each class. It explicitly uses class labels to compute class means $\boldsymbol{\mu}_c$ and the global mean $\boldsymbol{\mu}$. The projection of a sample is $\tilde{\mathbf{x}} = W^T \mathbf{x}$.

Training Objective

The PCA objective is to **minimize the average reconstruction error**:

$$P^* = \arg \min_P \frac{1}{K} \sum_{i=1}^K \|\mathbf{x}_i - PP^T \mathbf{x}_i\|^2, \quad \text{s.t. } P^T P = I$$

This is equivalent to **maximizing the projected variance**:

$$\tilde{L}(P) = \text{Tr}(P^T C P)$$

where $C = \frac{1}{K} \sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T$ is the empirical covariance matrix. The constraint $P^T P = I$ ensures orthonormality of the components.

The LDA objective is to **maximize the Fisher ratio** — the ratio of between-class variability to within-class variability. **For the multi-direction case, the criterion is:**

$$L = \text{Tr}(\tilde{S}_W^{-1} \tilde{S}_B) = \text{Tr}((W^T S_W W)^{-1} (W^T S_B W))$$

where the scatter matrices are:

$$S_B = \frac{1}{N} \sum_{c=1}^K n_c (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T, \quad S_W = \frac{1}{N} \sum_{c=1}^K \sum_{i=1}^{n_c} (\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)(\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)^T$$

Unlike PCA, LDA does *not* require W to be orthogonal.

Characteristics of the Directions

The table below summarizes the key properties of the directions found by each method:

Property	PCA Principal Components	LDA Discriminant Directions
Computed from	Eigenvectors of covariance matrix C	Eigenvectors of $S_W^{-1}S_B$
Ordering	By descending eigenvalue (= variance)	By descending eigenvalue (= Fisher ratio)
Orthogonality	Guaranteed ($P^T P = I$)	Not required
Max number of directions	Up to n (feature dimensionality)	At most $K - 1$ (number of classes minus one)
Label-awareness	No — may not be discriminant	Yes — explicitly maximizes class separation
Interpretability	Directions of highest variance	Directions of highest class separability

The $K - 1$ limit on LDA directions follows directly from the rank of S_B : since S_B is a sum of K rank-one matrices centered around the global mean, it has at most $K - 1$ non-zero eigenvalues. In contrast, PCA can extract up to n components, selected by retaining enough to explain a target fraction t of total variance:

$$\min_m m \quad \text{s.t.} \quad \frac{\sum_{i=1}^m \sigma_i}{\sum_{i=1}^n \sigma_i} \geq t$$

Use in Classification

PCA for classification is used as a *preprocessing step*. It reduces dimensionality before feeding data into a separate classifier (e.g., logistic regression, k-NN). Because PCA is unsupervised, there is no guarantee the retained components are discriminant — variance-maximizing directions and class-separating directions can differ significantly.

LDA for classification works directly. For a binary problem, the optimal direction is:

$$\mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$$

and the classification rule assigns sample $\mathbf{x}_t$ to class C_1 if $\mathbf{w}^T \mathbf{x}_t < t$, where the threshold is the midpoint of the projected class means:

$$t = \frac{m_1 + m_2}{2}, \quad m_c = \mathbf{w}^T \boldsymbol{\mu}_c$$

This is equivalent to a nearest-centroid classifier in the projected space. LDA implicitly assumes that within-class distributions are Gaussian with equal covariance, so the linear decision boundary is optimal under those assumptions.

A common practical strategy is to **apply PCA first, then LDA**: PCA removes noise and reduces the risk of S_W becoming singular (which happens

when the number of features is large relative to the number of samples), and LDA then finds the most discriminant subspace within the PCA-reduced space.

4 Generative Gaussian Models

Motivation. Generative Gaussian Models (GGM) address closed-set classification by explicitly modeling the data-generation process for each class. **Unlike discriminative models that learn decision boundaries directly, generative models assume a probabilistic class-conditional density and apply Bayes' theorem to compute posterior probabilities.** The fundamental assumption is that the likelihood of the data given a class follows a multivariate Gaussian distribution, allowing us to exploit the well-understood properties of Gaussians to derive tractable classification rules.

Probabilistic framework. Classification is performed by computing the posterior probability:

$$P(C_t = c | X_t = \mathbf{x}_t) = \frac{f_{X|C}(\mathbf{x}_t | c) \cdot P(C = c)}{\sum_{c'} f_{X|C}(\mathbf{x}_t | c') \cdot P(C = c')}$$

The classifier assigns $\mathbf{x}_t$ to the class with the highest posterior. The joint density factorizes as:

$$f_{X,C}(\mathbf{x}, c) = f_{X|C}(\mathbf{x} | c) \cdot P(C = c)$$

The class prior $P(C = c)$ is determined by the application; the class likelihood $f_{X|C}(\mathbf{x} | c)$ is learned from data.

Mathematical formulation. GGM assumes that each class likelihood is a multivariate Gaussian:

$$(X_t | C_t = c) \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

The log-likelihood per sample in class c is:

$$\log f_{X|C}(\mathbf{x} | c) = -\frac{D}{2} \log 2\pi - \frac{1}{2} \log |\boldsymbol{\Sigma}_c| - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_c)^T \boldsymbol{\Sigma}_c^{-1} (\mathbf{x} - \boldsymbol{\mu}_c)$$

Training via ML estimation. Parameters $\boldsymbol{\mu}_c$ and $\boldsymbol{\Sigma}_c$ are estimated by maximizing the log-likelihood class by class. The closed-form ML solutions are:

$$\boldsymbol{\mu}_c^* = \frac{1}{N_c} \sum_{i:c_i=c} \mathbf{x}_i, \quad \boldsymbol{\Sigma}_c^* = \frac{1}{N_c} \sum_{i:c_i=c} (\mathbf{x}_i - \boldsymbol{\mu}_c^*)(\mathbf{x}_i - \boldsymbol{\mu}_c^*)^T$$

where N_c is the number of samples in class c . **Each class is estimated independently: the sample mean is the class center, and the sample covariance captures the within-class spread.**

Inference: binary classification and LLR. For a binary decision, the classifier compares the log-likelihood ratio (LLR) to a threshold derived from the log-odds:

$$\text{llr}(\mathbf{x}_t) = \log \frac{f_{X|C}(\mathbf{x}_t|c_1)}{f_{X|C}(\mathbf{x}_t|c_0)} \geq -\log \frac{P(C=c_1)}{P(C=c_0)}$$

The LLR is a **quadratic function of $\mathbf{x}$** :

$$\text{llr}(\mathbf{x}) = \mathbf{x}^T A \mathbf{x} + \mathbf{x}^T \mathbf{b} + c$$

where the quadratic term $A = -\frac{1}{2}(\boldsymbol{\Lambda}_1 - \boldsymbol{\Lambda}_0)$ vanishes only when the two classes share the same covariance ($\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_0$). This produces a **quadratic decision boundary**, characteristic of QDA (Quadratic Discriminant Analysis).

Inference: multiclass classification. For $K > 2$ classes, assign the sample to:

$$c_t^* = \arg \max_h [\log f_{X|C}(\mathbf{x}_t|h) + \log P(C=h)]$$

The classifier outputs class-conditional log-likelihoods; the prior term $\log P(C=h)$ is application-dependent and can be adjusted at deployment time.

Model variants. GGM encompasses several important variants:

- **Full covariance (QDA):** Each class has its own full covariance matrix $\boldsymbol{\Sigma}_c$. Highly flexible but requires many samples; decision boundary is quadratic.

- **Tied covariance (LDA-like):** All classes share a common covariance Σ :

$$\Sigma^* = \frac{1}{N} \sum_c \sum_{i:c_i=c} (\mathbf{x}_i - \boldsymbol{\mu}_c^*)(\mathbf{x}_i - \boldsymbol{\mu}_c^*)^T$$

This reduces the number of parameters and produces a **linear decision boundary** (the quadratic term A cancels). More stable with limited data.

- **Naive Bayes Gaussian:** Each class has a diagonal covariance, assuming feature independence. Simple and computationally efficient, but assumes uncorrelated features.

Limitations. GGM assumes all data within a class are drawn from a Gaussian distribution — a strong assumption often violated in practice. **Estimating a full covariance matrix requires $\frac{D(D+1)}{2}$ parameters**, which becomes prohibitive in high dimensions and with limited data. The covariance matrix must be invertible; if $N < D$, standard ML estimation fails. A common solution is to apply PCA beforehand to reduce dimensionality and ensure invertibility, though care must be taken not to eliminate discriminant directions.

5 LDA vs Tied MVG

Both **LDA** (binary) and the **Tied-covariance MVG** classifier produce a **linear decision boundary**, and in fact coincide under equal priors. They reach the same rule from two different criteria: LDA from a *geometric* objective (Fisher ratio), Tied MVG from a *generative probabilistic* model.

Model Formulation, Training and Inference

LDA classifier. LDA maximizes the **Fisher ratio**, the ratio of between-class to within-class scatter:

$$\max_{\mathbf{w}} \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}, \quad S_B = \frac{1}{N} \sum_c n_c (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T, \quad S_W = \frac{1}{N} \sum_c \sum_i (\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)(\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)^T.$$

For the binary case the solution is in closed form, $\mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$, and inference projects the sample as $\mathbf{w}^T \mathbf{x}_t$.

Tied MVG classifier. Each class likelihood is Gaussian with a **single shared covariance**, $(X | C = c) \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma})$. Parameters are estimated by ML: the means are the class sample means, and $\boldsymbol{\Sigma}$ is the **pooled within-class covariance**

$$\boldsymbol{\Sigma}^* = \frac{1}{N} \sum_c \sum_{i:c_i=c} (\mathbf{x}_i - \boldsymbol{\mu}_c^*)(\mathbf{x}_i - \boldsymbol{\mu}_c^*)^T.$$

Inference is a Bayes decision: compare the log-likelihood ratio to the log-odds threshold.

Model assumptions. Both models make the *same* assumption: within-class data are Gaussian with a **common (tied) covariance**. The key structural identity is that LDA's within-class scatter S_W is the tied ML covariance $\boldsymbol{\Sigma}$ (up to the $1/N$ normalization).

The Relationship and the Decision Rules

For tied covariance the quadratic term of the MVG log-likelihood ratio, $A = -\frac{1}{2}(\boldsymbol{\Sigma}_1^{-1} - \boldsymbol{\Sigma}_0^{-1})$, **vanishes**, leaving a purely linear score with direction $\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$ — the **same direction** as LDA's $S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$. Setting the LLR to zero (equal priors) gives

$$(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^T \boldsymbol{\Sigma}^{-1} \mathbf{x} = \frac{1}{2}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^T \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 + \boldsymbol{\mu}_0) = \frac{m_1 + m_0}{2},$$

which is exactly LDA's midpoint threshold. **Under equal priors the two binary classifiers are identical** — same direction *and* same threshold — both reducing to a nearest-centroid (Mahalanobis) rule in the $\boldsymbol{\Sigma}$ -whitened space.

The decision rules differ only in how the threshold is set:

Aspect	LDA (binary)	Tied MVG (binary)
Objective	Maximize Fisher ratio (geometric)	Maximize per-class Gaussian likelihood (generative)
Score direction	$S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$	$\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$ — same since $\boldsymbol{\Sigma} \propto S_W$
Decision rule	$\mathbf{w}^T \mathbf{x}_t < t, \quad t = \frac{m_1 + m_2}{2}$	$\text{llr}(\mathbf{x}_t) \gtrless -\log \frac{P(c_1)}{P(c_0)}$
Threshold / priors	Midpoint; optimal only for equal priors	Log-odds; natively handles priors and costs
Boundary	Linear	Linear (quadratic term A cancels)

The **practical difference** is that the Tied MVG threshold is derived from the application priors (and can be shifted for costs), whereas LDA’s midpoint is optimal only when the priors are equal — if they differ, LDA’s threshold must be adjusted by hand.

LDA for Multiclass Dimensionality Reduction

For multiclass problems LDA is used as a **dimensionality-reduction** technique: it collects the m most discriminant directions in W , maximizing

$$\max_W \text{Tr}((W^T S_W W)^{-1} (W^T S_B W)) .$$

Limitations. Since S_B has rank at most $K - 1$, LDA can extract **at most** $K - 1$ **directions** regardless of the feature dimension — a hard ceiling that discards potentially useful directions when K is small. It also assumes equal-covariance Gaussian classes (often violated), and S_W becomes singular when $N < D$ (a common remedy is applying PCA first). In contrast, the Tied MVG handles the multiclass case *directly* via per-class posteriors, with no $K - 1$ ceiling.

6 Logistic Regression

Logistic Regression is a **discriminative** model for binary classification: instead of modelling the class-conditional densities, it models the **class posterior** $P(C|X)$ directly. It assumes the log-posterior-ratio is a **linear** function of the input, so the posterior is a sigmoid of a linear score.

Classification Rule and Probabilistic Interpretation

Assuming a linear decision rule parametrized by $(\mathbf{w}, b)$, the posterior for class h_1 is the **sigmoid** of a linear score:

$$P(C=h_1 | \mathbf{x}, \mathbf{w}, b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} = \sigma(\mathbf{w}^T \mathbf{x} + b), \quad \sigma(t) = \frac{1}{1 + e^{-t}} .$$

The **score** $s = \mathbf{w}^T \mathbf{x} + b$ has a precise probabilistic meaning: it is the **log-posterior-ratio**

$$s = \mathbf{w}^T \mathbf{x} + b = \log \frac{P(C=1 | \mathbf{x})}{P(C=0 | \mathbf{x})}.$$

The **classification rule** is therefore $s \geq 0$: a **linear hyperplane orthogonal to $\mathbf{w}$** . The magnitude $|s|$ relates to the (signed) distance from the separating surface, i.e. how confident the prediction is. Being discriminative, the model does *not* need an explicit model of the feature density $f_X(\mathbf{x})$: in the joint likelihood $f_{X,C} = P(C | X, \theta) f_X(\mathbf{x})$ the term f_X does not depend on $\theta = (\mathbf{w}, b)$ and is dropped.

Parameter Estimation and the Training Objective

Parameters are estimated by **Maximum Likelihood** on the labels (i.i.d. samples). Writing $y_i = \sigma(\mathbf{w}^T \mathbf{x}_i + b)$, each label is Bernoulli, $C_i | \mathbf{x}_i \sim \text{Ber}(y_i)$, so

$$\ell(\mathbf{w}, b) = \sum_{i=1}^n [c_i \log y_i + (1 - c_i) \log(1 - y_i)].$$

The same objective admits **three equivalent interpretations**:

- (1) **Maximum Likelihood** of the observed labels.
- (2) **Minimum cross-entropy**. Minimizing $J = -\ell = \sum_i H(c_i, y_i)$ minimizes the average **cross-entropy** $H(P, Q) = \mathbb{E}_P[-\log Q]$ between the empirical label distribution $P = \text{Ber}(c_i)$ (an evaluator who knows the true label) and the predicted $Q = \text{Ber}(y_i)$; it is minimized when $Q = P$.
- (3) **Empirical risk minimization**. Using $z_i = 2c_i - 1 \in \{-1, +1\}$ and $s_i = \mathbf{w}^T \mathbf{x}_i + b$, the objective becomes a sum of per-sample losses:

$$J(\mathbf{w}, b) = \sum_{i=1}^n \log(1 + e^{-z_i s_i}) = \sum_{i=1}^n \ell(-z_i s_i), \quad \ell(t) = \log(1 + e^{-t}).$$

The loss $\ell(-z_i s_i)$ is the **cost** of the prediction: when prediction and label agree ($z_i s_i > 0$) the cost decays (asymptotically) to 0 as $|s_i|$ grows; when they disagree it grows (asymptotically) *linearly* in $|s_i|$. This casts LR as the minimization of an **empirical risk** $R(\theta) = \sum_i \ell(\theta, \mathbf{x}_i, z_i)$. The solution has **no closed form** and is found with a numerical solver from the loss and its gradient.

Regularization

If the classes are **linearly separable**, the loss has *no minimum*: it can be driven to its infimum $\inf J = 0$ by sending $\|\mathbf{w}\| \rightarrow \infty$, so the parameters do not converge. We add a **norm penalty** (regularization term), obtaining **regularized LR**:

$$R(\mathbf{w}, b) = \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_{i=1}^n \log(1 + e^{-z_i s_i}),$$

an instance of **regularized risk minimization** $R = \Phi(\mathbf{w}) + \frac{1}{n} \sum_i \ell$. The hyper-parameter λ trades separation against simplicity (too large $\Rightarrow$ small norm but poor separation; too small $\Rightarrow$ overfitting) and is selected by **cross-validation** (it cannot be tuned by minimizing R , which would give $\lambda = 0$). Unlike the unregularized model, regularized LR is **not invariant** to linear transformations of the features, so pre-processing (centering, variance standardization, whitening, length normalization) can help.

Score, Priors and Recalibration

Since the parameters are trained on the labels, the score reflects the **empirical training prior**. To use it as a **log-likelihood ratio** for an arbitrary application, subtract the empirical prior log-odds:

$$s_{llr} = \mathbf{w}^T \mathbf{x} + b - \log \frac{n_T}{n_F}, \quad \text{decide } s_{llr} \gtrless \log \frac{\pi_T}{1 - \pi_T}.$$

If the application prior π_T is known *before* training, one can directly optimize **prior-weighted LR**, which reweights the two class losses by π_T/n_T and $(1 - \pi_T)/n_F$; standard LR is the special case with the empirical prior.

Non-linear Decision Functions and the Multi-class Case

Non-linearity. LR can be trained on an **expanded feature space** $\phi(\mathbf{x})$: a linear rule $\mathbf{w}^T \phi(\mathbf{x}) + b$ in the expanded space corresponds to a **non-linear** (e.g. quadratic) boundary in the original space. The cost is that $\dim \phi$ can grow very quickly, raising computation and overfitting risk.

Multiclass (softmax). For K classes the posteriors take the **softmax** form

$$P(C=k | \mathbf{x}) = \frac{e^{\mathbf{w}_k^T \mathbf{x} + b_k}}{\sum_{j=1}^K e^{\mathbf{w}_j^T \mathbf{x} + b_j}},$$

and training minimizes the multiclass cross-entropy (**softmax loss**) $-\sum_i \sum_k z_{ik} \log y_{ik}$ with 1-of- K labels. The model is **over-parametrized** (adding a constant vector to all $\mathbf{w}_j$ leaves it unchanged); a regularization term is added as in the binary case.

7 Logistic Regression vs Generative Gaussian Models

Logistic Regression (LR) and the **Generative Gaussian Models** (MVG) are the prototypical **discriminative** and **generative** classifiers. They reach a probabilistic decision in opposite ways, yet share the same family of decision functions.

What Each Model Estimates

The **generative** MVG models the *joint* distribution through the class-conditional densities and the prior, $f(\mathbf{x} | c) P(c)$, and obtains the posterior by Bayes. The **discriminative** LR models the posterior $P(c | \mathbf{x})$ *directly*, and does **not** need a model of the feature density $f_X(\mathbf{x})$. Consequently MVG is estimated in **closed form** (per-class ML means and covariances), whereas LR has **no closed form** and minimizes the cross-entropy / empirical risk numerically.

The Shared Linear Form (LR $\leftrightarrow$ Tied MVG)

The key connection is that the **Tied MVG** has a **linear** log-posterior-ratio

$$s = \mathbf{w}^T \mathbf{x} + b, \quad \mathbf{w} \propto \Sigma^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0),$$

which is *exactly* the parametric form LR assumes a priori. So **LR (with linear features)** and the **Tied MVG** share the same hypothesis class

— linear log-odds — and differ only in *how* they pick $(\mathbf{w}, b)$: MVG **generatively** (Gaussian ML, then derive the posterior), LR **discriminatively** (optimize the conditional likelihood, ignoring the densities). Likewise, the full MVG (QDA) gives a quadratic boundary, matched by LR trained on a quadratic feature expansion $\phi(\mathbf{x})$.

Assumptions, Priors and Trade-offs

MVG assumes **Gaussian** class-conditionals (a strong assumption): when it holds, MVG is more **data-efficient**; when it is violated, LR — which only assumes a linear log-odds — is often more robust. On **priors**, the generative model separates likelihood from prior, so priors and costs can be changed at deployment by moving the threshold (the LLR is Bayes-ready); LR instead embeds the **empirical training prior** in its score and requires **recalibration** ($s_{llr} = s - \log \frac{n_T}{n_F}$) or **prior-weighted** training. Finally, a full MVG needs $\frac{D(D+1)}{2}$ covariance parameters per class (hence $N > D$, often PCA first), while linear LR needs only $D + 1$ and scales better, at the price of a linear boundary unless features are expanded.

Aspect	Logistic Regression	Generative Gaussian (MVG / Tied)
Type	Discriminative: models $P(C \mathbf{x})$ directly	Generative: models $f(\mathbf{x} c)P(c)$, then Bayes
Training	ML / cross-entropy, numerical (no closed form)	ML in closed form, per class
Assumptions	Linear log-odds; no model of f_X	Gaussian class-conditionals
Decision rule	$s \gtrless 0$; linear (quadratic if $\phi(\mathbf{x})$)	$\text{llr} \gtrless -\log \frac{P(c_1)}{P(c_0)}$; quadratic (QDA), linear if tied
Priors	Score tied to empirical prior $\Rightarrow$ recalibrate	Prior separated $\Rightarrow$ threshold set natively
Parameters	$D + 1$ (linear)	full $\frac{D(D+1)}{2}$ per class; tied: one Σ
Strength	Robust to non-Gaussian features; few parameters	Data-efficient if Gaussian holds; LLR Bayes-ready

8 Logistic Regression vs LDA

LDA (binary) and **Logistic Regression** (LR) both produce a **linear** binary classifier — a decision based on $s = \mathbf{w}^T \mathbf{x} + b$ with a boundary that is a hyperplane orthogonal to $\mathbf{w}$ — but they choose the separating rule from **different criteria**: LDA from a geometric objective, LR from a discriminative (probabilistic) one.

Objective and Training

LDA maximizes the **Fisher ratio**, the ratio of between- to within-class scatter,

$$\max_{\mathbf{w}} \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}, \quad \mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1),$$

a purely **geometric** criterion solved in **closed form** (a generalized eigenproblem). **LR** instead minimizes the **cross-entropy** / **empirical risk**

$$J(\mathbf{w}, b) = \sum_i \log(1 + e^{-z_i(\mathbf{w}^T \mathbf{x}_i + b)}),$$

a **discriminative** criterion with **no closed form**, solved numerically. Both live in the same hypothesis class (linear log-odds, which a common-covariance Gaussian implies); they only disagree on which hyperplane to select.

Assumptions, Thresholds and Extensions

Assumptions. LDA assumes **Gaussian classes with a common covariance**; LR assumes only a **linear log-odds** with no model of the feature density, making it more robust when the Gaussian assumption fails. **Threshold/priors.** LDA classifies against the **midpoint** threshold $t = \frac{m_1 + m_2}{2}$ (optimal under equal priors); LR's score reflects the **empirical training prior** and can be recalibrated to a log-likelihood ratio for arbitrary applications. **Non-linearity.** LR extends to non-linear boundaries via a feature expansion $\phi(\mathbf{x})$; LDA, being geometric/eigen-based, does not extend naturally. **Dual role / multiclass.** LDA is also a **dimensionality-reduction** technique — at most $K - 1$ discriminant directions, then a downstream classifier — whereas LR is purely a classifier and handles multiclass directly via **softmax**.

Aspect	Logistic Regression	LDA (binary)
Objective	Min cross-entropy / empirical risk (discriminative)	Max Fisher ratio (geometric)
Training	Numerical, no closed form	Closed form (eigenproblem / $S_W^{-1} \Delta \boldsymbol{\mu}$)
Assumptions	Linear log-odds; no model of f_X	Gaussian classes, common covariance
Decision rule	$s \geq 0$ (recalibratable to LLR)	$\mathbf{w}^T \mathbf{x} \leq t = \frac{m_1 + m_2}{2}$ (midpoint)
Dual use	Classifier only	Also dimensionality reduction ($\leq K - 1$ dir.)
Non-linearity	Yes, via $\phi(\mathbf{x})$	No (geometric)
Multiclass	Softmax (direct)	Dim. reduction $\leq K - 1$ + downstream classifier

9 Support Vector Machines

The **Support Vector Machine** (SVM) is a **discriminative, non-probabilistic** binary classifier that gives a **geometric interpretation of the regularization term** of Logistic Regression. Among the infinitely many hyperplanes that separate two linearly separable classes, it selects the one with the **largest margin**, and through the **kernel trick** it achieves non-linear separation *without* an explicit feature expansion. Its output **cannot be interpreted as a class posterior**.

Maximum-Margin Hyperplane

Let $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ be the separating surface and encode labels as $z_i \in \{+1, -1\}$ ($z_i = +1$ for $\mathcal{H}_T$, $z_i = -1$ for $\mathcal{H}_F$). The distance of a point from the hyperplane is $d(\mathbf{x}_i) = \frac{|f(\mathbf{x}_i)|}{\|\mathbf{w}\|}$, and the **margin** is the distance of the closest point. The maximum-margin hyperplane maximizes the minimum distance:

$$\mathbf{w}^*, b^* = \arg \max_{\mathbf{w}, b} \min_i \frac{|z_i(\mathbf{w}^T \mathbf{x}_i + b)|}{\|\mathbf{w}\|}.$$

Since the classes are separable, an optimal solution has $z_i(\mathbf{w}^T \mathbf{x}_i + b) > 0 \forall i$, so the absolute value can be dropped: $\arg \max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|} \min_i [z_i(\mathbf{w}^T \mathbf{x}_i + b)]$.

Motivation. Logistic Regression picks the hyperplane maximizing the posteriors, but given any separating $\mathbf{w}$ one can lower the loss by keeping the orientation and *growing* $\|\mathbf{w}\|$ (posteriors $\rightarrow 0/1$). The margin gives a well-defined criterion instead.

Canonical Form and Primal Problem

The objective is **invariant under rescaling** $(\mathbf{w}, b) \rightarrow (\alpha \mathbf{w}, \alpha b)$, so each solution is an equivalence class. We fix the representative by requiring $\min_i z_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$, hence $z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \forall i$. The problem becomes the **primal SVM**:

$$\arg \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1 \dots n.$$

At the optimum the constraint is **active** ($\min_i = 1$): a solution with $\min_i z_i(\mathbf{w}^T \mathbf{x}_i + b) = \psi > 1$ could be rescaled by $1/\psi$, satisfying all constraints with smal-

ler norm. This is a **convex quadratic programming** problem (convex objective, convex feasible set), so every local minimum is global.

Lagrangian, Dual Problem and Support Vectors

Introducing multipliers $\alpha_i \geq 0$ for each constraint,

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1].$$

Setting $\nabla_{\mathbf{w}} L = 0$ and $\partial L / \partial b = 0$ gives $\mathbf{w} = \sum_i \alpha_i z_i \mathbf{x}_i$ and $\sum_i \alpha_i z_i = 0$. Substituting yields the **dual problem**:

$$\begin{aligned} \max_{\boldsymbol{\alpha}} \quad & \boldsymbol{\alpha}^T \mathbf{1} - \frac{1}{2} \boldsymbol{\alpha}^T \mathbf{H} \boldsymbol{\alpha}, \quad H_{ij} = z_i z_j \mathbf{x}_i^T \mathbf{x}_j, \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i z_i = 0. \end{aligned}$$

Crucially, $\mathbf{H}$ depends on the data **only through the dot products** $\mathbf{x}_i^T \mathbf{x}_j$: this is what enables kernels. **Weak/strong duality**: $L_D(\boldsymbol{\alpha}) \leq L_P(\mathbf{w}, b)$ for any feasible pair, and the **duality gap** $L_P - L_D \geq 0$ vanishes at the optimum ($L_D(\boldsymbol{\alpha}^*) = L_P(\mathbf{w}^*, b^*)$).

A solution is optimal iff it satisfies the **KKT conditions**: stationarity in $\mathbf{w}$ and b , primal feasibility $z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0$, dual feasibility $\alpha_i \geq 0$, and **complementary slackness** $\alpha_i [z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0$. The last one implies that points strictly outside the margin ($z_i(\mathbf{w}^T \mathbf{x}_i + b) > 1$) have $\alpha_i = 0$, while $\alpha_i \neq 0$ means the point lies **on the margin**: these are the **support vectors**. Non-support vectors do not affect the separating surface; b is recovered from the KKT conditions once $\boldsymbol{\alpha}$ is known.

Soft Margin (Non-Separable Classes)

When classes are not separable, some constraints are always violated. We introduce **slack variables** $\xi_i \geq 0$ relaxing the constraints to $z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$. Choosing $\sigma = 1$, $F(u) = u$ in the general penalty $\frac{1}{2} \|\mathbf{w}\|^2 + CF(\sum_i \xi_i^\sigma)$ gives the **soft-margin** objective:

$$\min_{\mathbf{w}, b, \boldsymbol{\xi}} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The ξ_i are a **penalty** (not a count): points inside the margin have $\xi_i > 0$, mis-classified points $\xi_i > 1$, and $\sum_i \xi_i$ is an **upper bound on the number of errors**. The constant C trades off margin width against training errors; the result is a **soft-margin hyperplane**. The dual is unchanged except for a **box constraint** $0 \leq \alpha_i \leq C$ (besides $\sum_i \alpha_i z_i = 0$).

Hinge Loss and Comparison with Logistic Regression

On correctly classified points $\xi_i = 0$, otherwise $\xi_i = 1 - z_i(\mathbf{w}^T \mathbf{x}_i + b)$, so the constraints are absorbed into the unconstrained primal

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max[0, 1 - z_i(\mathbf{w}^T \mathbf{x}_i + b)],$$

where $f(s) = \max(0, 1 - s)$ is the **hinge loss**. In regularized-risk form SVM and LR share the same structure and differ *only* in the per-sample loss:

$$\text{SVM: } \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_i \max[0, 1 - z_i s_i], \quad \text{LR: } \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_i \log(1 + e^{-z_i s_i}),$$

with $s_i = \mathbf{w}^T \mathbf{x}_i + b$. The hinge loss is **exactly zero** for points correctly classified beyond the margin (giving sparsity / support vectors), while the log-loss is always positive.

Inference and the Kernel Trick

The score can be computed in two ways:

$$\text{primal: } s(\mathbf{x}_t) = \mathbf{w}^T \mathbf{x}_t + b \quad (O(D)), \quad \text{dual: } s(\mathbf{x}_t) = \sum_{i: \alpha_i > 0} \alpha_i z_i \mathbf{x}_i^T \mathbf{x}_t + b \quad (O(\#SV)).$$

The **decision rule** is the sign $s(\mathbf{x}_t) \geq 0$. Both forms depend on the data **only through dot products**. A **non-linear** mapping $\Phi: \mathbb{R}^D \rightarrow \mathcal{H}$ enters the dual and the score only as $\Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)$, so we replace it by a **kernel** $k(\mathbf{x}_1, \mathbf{x}_2) = \Phi(\mathbf{x}_1)^T \Phi(\mathbf{x}_2)$ without ever building Φ :

$$H_{ij} = z_i z_j k(\mathbf{x}_i, \mathbf{x}_j), \quad s(\mathbf{x}_t) = \sum_{i: \alpha_i > 0} \alpha_i z_i k(\mathbf{x}_i, \mathbf{x}_t) + b.$$

A linear separation in the expanded space is a **non-linear** separation in the original space, and the dual cost depends only on N , not on $\dim \mathcal{H}$. Common kernels: the **polynomial** kernel $k(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1^T \mathbf{x}_2 + 1)^d$ (e.g. $d = 2$ from $\Phi(\mathbf{x}) = [\text{vec}(\mathbf{x}\mathbf{x}^T); \sqrt{2}\mathbf{x}; 1]$) and the **RBF** / **Gaussian** kernel $k(\mathbf{x}_1, \mathbf{x}_2) = e^{-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2}$, associated to an **infinite-dimensional** mapping (γ small $\Rightarrow$ wide kernel, a support vector influences far points; γ large $\Rightarrow$ narrow, local influence). **Mercer's condition** is a *sufficient* condition for k to be a dot product in some space: k symmetric and $\int k(\mathbf{u}, \mathbf{v})g(\mathbf{u})g(\mathbf{v})d\mathbf{u}d\mathbf{v} > 0$ for all square-integrable g . It does not say how to build kernels; one uses known kernels or composition rules (e.g. a sum of kernels is a kernel).

Practical Considerations and Limitations

The kernel, its hyper-parameters and C are selected by **cross-validation**; sometimes a linear classifier suffices (high-dimensional, almost separable features). SVM training is **not invariant under affine transformations**, so centering and whitening usually help. SVM scores have **no probabilistic interpretation**: a post-processing/**calibration** step is needed to obtain posteriors. For **class imbalance** (training prior $\neq$ target prior) one uses class-dependent costs C_i ,

$$\arg \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^n C_i [1 - z_i(\mathbf{w}^T \mathbf{x}_i + b)]_+, \quad C_T = C \frac{\pi_T}{\pi_T^{emp}}, \quad C_F = C \frac{\pi_F}{\pi_F^{emp}},$$

simulating the application priors at training time (still without a probabilistic score). Finally, SVM is natively binary and **hard to extend to multiclass**. Kernels are not exclusive to SVMs: any method depending only on dot products (e.g. Logistic Regression) can be kernelized.

[INTEGRAZIONE TEORICA]

Perché i support vector sono solo i punti sul margine. La complementary slackness $\alpha_i [z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0$ impone che, per ogni punto, almeno uno tra α_i e il *gap* $z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1$ sia nullo. Quindi un punto strettamente fuori dal margine ($\text{gap} > 0$) deve avere $\alpha_i = 0$ e, dato che $\mathbf{w} = \sum_i \alpha_i z_i \mathbf{x}_i$, *non contribuisce* alla soluzione. Solo i punti con *gap* nullo (sul margine) possono avere $\alpha_i > 0$: sono i support vector, gli unici che definiscono l'iperpiano.

10 Logistic Regression vs SVM

Logistic Regression (LR) and the **Support Vector Machine (SVM)** are the two prototypical **discriminative, linear** classifiers cast as **regularized risk minimization**. They learn the same kind of score $s = \mathbf{w}^T \mathbf{x} + b$ and classify by its sign; they differ essentially in *one ingredient* — the per-sample loss — and in whether the output is probabilistic.

The Shared Risk-Minimization Form

Both minimize an objective of the form $\frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_i \ell(z_i s_i)$, with $z_i \in \{\pm 1\}$ and $s_i = \mathbf{w}^T \mathbf{x}_i + b$, differing **only in the loss**:

LR: $\ell(z_i s_i) = \log(1 + e^{-z_i s_i})$, SVM: $\ell(z_i s_i) = \max(0, 1 - z_i s_i)$ (hinge).

Neither models the feature density $f_X(\mathbf{x})$. The **regularization term** $\frac{\lambda}{2} \|\mathbf{w}\|^2$ plays the same role in both, but with complementary readings: in LR it is needed because on **separable** data the log-loss has no minimum (driving $\|\mathbf{w}\| \rightarrow \infty$); in SVM the *same* term **is** the objective, and minimizing $\frac{1}{2} \|\mathbf{w}\|^2$ subject to $z_i s_i \geq 1$ is exactly **maximizing the margin**. SVM is thus the **geometric interpretation** of LR's regularization.

Loss Shape, Support Vectors and Probabilities

The hinge loss is **exactly zero** for points correctly classified beyond the margin: only the points on/inside the margin (the **support vectors**, $\alpha_i \neq 0$) determine the solution, so SVM is **sparse** in the training set. The log-loss is **always positive**, so in LR **every** sample contributes. The other central difference is the **output**: LR returns a calibrated **posterior** $\sigma(s)$ with s a log-posterior-ratio, whereas the SVM score has **no probabilistic meaning** and needs an explicit **calibration** step before it can be used as an LLR.

Optimization, Non-linearity, Priors and Multiclass

LR is solved by **direct numerical minimization** of the primal (loss + gradient); SVM is a **convex QP**, usually solved through its **dual** (Lagrangian + KKT conditions), which both exposes the support vectors and enables kernels. **Non-linearity** is reached differently: LR uses an **explicit** feature expansion $\phi(\mathbf{x})$, SVM uses the **kernel trick** $k(\mathbf{x}_i, \mathbf{x}_j)$ (implicit, possibly infinite-dimensional) — though kernels are not exclusive to SVM and LR can be kernelized too. On **priors**, neither score natively matches the application prior: LR is **recalibrated** ($s_{lrr} = s - \log \frac{n_T}{n_F}$) or trained prior-weighted, while SVM rebalances through class-dependent costs C_T, C_F . Finally, LR extends to **multiclass** naturally via **softmax**, whereas SVM is natively binary and hard to extend.

Aspect	Logistic Regression	Support Vector Machine
Objective	$\frac{\lambda}{2} \ \mathbf{w}\ ^2 + \frac{1}{n} \sum \log(1 + e^{-z_i s_i})$	$\frac{\lambda}{2} \ \mathbf{w}\ ^2 + \frac{1}{n} \sum \max(0, 1 - z_i s_i)$
Loss (per sample)	log / softplus (always > 0)	hinge (zero beyond the margin)
Training	ML / cross-entropy, numerical solver	convex QP, dual + KKT
Output / score	posterior $\sigma(s)$; s is a log-odds	sign of s ; no probabilistic meaning
Decision rule	$s \gtrless 0$; Bayes threshold after recalibration	$s \gtrless 0$; no native Bayes threshold
Non-linearity	explicit feature expansion $\phi(\mathbf{x})$	implicit kernel k (even ∞ -dim)
Sparsity	no (all samples contribute)	yes (only support vectors)
Multiclass	natural (softmax)	hard (binary by nature)

11 Generative Gaussian Models vs SVM

The **Generative Gaussian Models** (MVG) and the **Support Vector Machine** (SVM) sit at opposite ends of the classifier spectrum: a **generative, probabilistic** model versus a **discriminative, geometric, non-probabilistic** one. They can produce the *same* decision boundaries, but obtain and interpret them in completely different ways.

Same Boundaries, Opposite Constructions

The **Tied** MVG has a **linear** log-posterior-ratio $s = \mathbf{w}^T \mathbf{x} + b$ with $\mathbf{w} \propto \Sigma^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$, the same *form* as the linear SVM; the **full** MVG (QDA) gives

a **quadratic** boundary, matched by an SVM with a degree-2 **polynomial kernel**. So the two can span the same family of surfaces. The difference is *how*: MVG models the class-conditional densities $f(\mathbf{x}|c)P(c)$ and gets the boundary through **Bayes**, estimating $\mathbf{w}$ from **ML Gaussian statistics**; SVM never models a density and obtains the boundary by **maximizing the margin**.

Probabilities, Training and What the Data Are Used For

MVG produces a **log-likelihood ratio** / posterior, **Bayes-ready**: the decision rule is $\text{llr} \gtrless -\log \frac{P(c_1)}{P(c_0)}$ and priors/costs can be changed at deployment by moving the threshold. The SVM score has **no probabilistic interpretation**: it needs **calibration** and has no native Bayes threshold; imbalance is handled through class-dependent costs C_T, C_F . **Training** also diverges: MVG is **closed-form** (per-class means and covariances), SVM is a **convex QP** (dual + KKT, no closed form). Finally, MVG uses **aggregate statistics of all points**, whereas the SVM solution depends **only on the support vectors** and on **dot products**, hence is kernelizable.

Assumptions, Non-linearity and Multiclass

MVG makes a **strong Gaussian assumption** on the class-conditionals (data-efficient when it holds, requiring Σ invertible, $N > D$, often PCA first); SVM makes **no density assumption** and is more robust to non-Gaussian features. For **non-linear** boundaries MVG uses a full covariance (quadratic, with $\frac{D(D+1)}{2}$ parameters per class), while SVM uses the **kernel trick** (even infinite-dimensional, without building Φ). MVG is **natively multiclass** (the arg max of the log-posterior over K classes), whereas SVM is binary by nature and **hard to extend**.

Aspect	Generative Gaussian (MVG / Tied)	Support Vector Machine
Type	Generative: $f(\mathbf{x} c)P(c)$, then Bayes	Discriminative, non-probabilistic
Objective	ML of Gaussian class-conditionals	max margin / hinge $+\frac{1}{2}\ \mathbf{w}\ ^2$
Assumptions	Gaussian class-conditionals; Σ invertible	none on the feature density
Training	closed form, per class	convex QP (dual, KKT)
Score	LLR $\log \frac{f(\mathbf{x} c_1)}{f(\mathbf{x} c_0)}$; posterior via Bayes	$\sum_{SV} \alpha_i z_i k(\mathbf{x}_i, \mathbf{x}) + b$; no posterior
Decision rule	$\text{llr} \gtrless -\log \frac{P(c_1)}{P(c_0)}$; quadratic (linear if tied)	$s \gtrless 0$; linear or kernel-nonlinear
Priors	separated $\Rightarrow$ threshold/costs native	no native prior; costs C_T, C_F
Multiclass	native ($\arg \max$ log-posterior)	hard (binary by nature)

12 Gaussian Mixture Models

Motivation. A **Gaussian Mixture Model (GMM)** is a density model obtained as a **weighted combination of Gaussians**. By combining several simple Gaussian components it can approximate **any sufficiently regular distribution** to a desired degree, provided enough data. GMMs are used for **density estimation**, **clustering** (a generalization of K-means), and as **generative classifiers** (one GMM per class). The starting observation is that, in a Gaussian classifier, the marginal density $f_X(x) = \sum_c \pi_c \mathcal{N}(x; \boldsymbol{\mu}_c, \Sigma_c)$ is *already* a GMM.

Mathematical formulation. A K -component GMM is:

$$f_X(x) = \sum_{c=1}^K w_c \mathcal{N}(x; \boldsymbol{\mu}_c, \Sigma_c), \quad w_c \geq 0, \quad \sum_{c=1}^K w_c = 1$$

The constraint $\sum_c w_c = 1$ follows from requiring $\int f_X(x) dx = 1$. The parameters are $\theta = (\mathcal{M}, \mathcal{S}, \mathbf{w})$ with $\mathcal{M} = \{\boldsymbol{\mu}_c\}$, $\mathcal{S} = \{\Sigma_c\}$, $\mathbf{w} = \{w_c\}$.

Latent-variable interpretation. A GMM is the **marginal** of a joint distribution over a sample X_i and a hidden **cluster** variable C_i :

$$f_{X_i}(x_i) = \sum_{c=1}^K f_{X_i|C_i}(x_i|c) P(C_i = c) = \sum_{c=1}^K w_c \mathcal{N}(x_i; \boldsymbol{\mu}_c, \Sigma_c)$$

with categorical prior $P(C_i = c) = w_c$ and conditional $f_{X_i|C_i}(x_i|c) = \mathcal{N}(x_i; \boldsymbol{\mu}_c, \Sigma_c)$. The cluster membership is a **latent (hidden) variable**: it is not observed but governs the data-generation process.

Likelihood and ill-posed ML. For an i.i.d. dataset $\mathcal{D} = \{x_1, \dots, x_N\}$ (treated as **unlabeled**), the log-likelihood is:

$$\ell(\theta) = \sum_{i=1}^N \log \sum_{c=1}^K w_c \mathcal{N}(x_i; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

The **log of a sum** has no closed-form maximizer. Moreover, ML for GMMs is **ill-posed**: with at least two components the likelihood is **unbounded above** (a component can collapse onto a single point, $\boldsymbol{\Sigma}_c \rightarrow 0$), so degenerate solutions exist. Combined with heuristics, ML still yields good density estimates.

Responsibilities and hard assignment. Given θ , the **responsibility** (cluster posterior) of component c for sample x_i is:

$$\gamma_{c,i} = P(C_i = c | X_i = x_i) = \frac{w_c \mathcal{N}(x_i; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)}{\sum_{c'} w_{c'} \mathcal{N}(x_i; \boldsymbol{\mu}_{c'}, \boldsymbol{\Sigma}_{c'})}$$

A first, crude approach assigns each point to $\hat{c}_i = \arg \max_c \gamma_{c,i}$ (**hard clustering**) and re-estimates parameters by ML as if labels were known. This ignores uncertainty when clusters overlap and does **not** maximize the observed-data likelihood.

K-means as a special case. If we fix $\boldsymbol{\Sigma}_c = I$ and $w_c = 1/K$, hard assignment reduces to:

$$\hat{c}_i = \arg \max_c P(C_i = c | x_i) = \arg \min_c \|x_i - \boldsymbol{\mu}_c\|^2$$

i.e. assign each point to the nearest centroid and re-estimate centroids — the **K-means** algorithm. K-means is therefore a special case of (hard-assignment, isotropic) GMM training.

Training via EM. Soft assignments are handled by the **Expectation-Maximization (EM)** algorithm, the general method for ML estimation when the likelihood is a marginal $f_X(x) = \int f_{X,H}(x, h) dh$ over a latent variable H . Introducing a distribution $Q(h)$, the log-likelihood decomposes as:

$$\log f_X(x; \theta) = \mathcal{L}_h(Q, \theta) + D_{KL}(Q \| f_{H|X}(h|x; \theta)), \quad D_{KL} \geq 0$$

so $\mathcal{L}_h$ is a **lower bound (ELBO)** on the log-likelihood. EM iterates:

- **E-step:** maximize the bound w.r.t. Q by setting $Q^t(h) = f_{H|X}(h|x; \theta^t)$, which makes $D_{KL} = 0$ (the bound becomes tight). It computes the **auxiliary function** $\mathcal{Q}(\theta, \theta^t) = \mathbb{E}_{f_{H|X}(h|x; \theta^t)}[\log f_{X,H}(x, h; \theta)]$.
- **M-step:** maximize the bound w.r.t. θ : $\theta^{t+1} = \arg \max_{\theta} \mathcal{Q}(\theta, \theta^t)$.

Because the E-step makes the bound tight and the M-step never decreases it, the likelihood is **monotonically non-decreasing**: $\log f_X(x; \theta^{t+1}) \geq \log f_X(x; \theta^t)$. EM converges to a **stationary point** (local maximum / saddle) that depends on the initialization.

EM for GMM. The latent variables are the cluster assignments $h = \{C_1, \dots, C_N\}$, with joint $f_{X_i, C_i}(x_i, c) = w_c \mathcal{N}(x_i; \mu_c, \Sigma_c)$. The two steps become:

- **E-step:** compute $\gamma_{c,i} = P(C_i = c|x_i; \theta^t)$. The auxiliary function is

$$\mathcal{Q}(\theta, \theta^t) = \sum_{i=1}^N \sum_{c=1}^K \gamma_{c,i} [\log \mathcal{N}(x_i; \mu_c, \Sigma_c) + \log w_c]$$

- **M-step:** maximize w.r.t. θ subject to $\sum_c w_c = 1$:

$$\mu_c^* = \frac{\sum_i \gamma_{c,i} x_i}{\sum_i \gamma_{c,i}}, \quad \Sigma_c^* = \frac{\sum_i \gamma_{c,i} (x_i - \mu_c^*)(x_i - \mu_c^*)^T}{\sum_i \gamma_{c,i}}, \quad w_c^* = \frac{\sum_i \gamma_{c,i}}{\sum_{i,c'} \gamma_{c',i}}$$

These are the **soft** versions of the hard-assignment ML updates (membership weights $\gamma_{c,i}$ replace 0/1 labels). Defining the **zero, first and second order statistics** $N_c = \sum_i \gamma_{c,i}$, $F_c = \sum_i \gamma_{c,i} x_i$, $S_c = \sum_i \gamma_{c,i} x_i x_i^T$, the updates read $\mu_c^* = F_c/N_c$, $\Sigma_c^* = S_c/N_c - \mu_c^* \mu_c^{*T}$, $w_c^* = N_c/N$.

Inference: classification. For a closed-set task we fit a GMM with K_c components to each class and apply Bayes:

$$f_{X_t|C_t}(x_t|c) = \sum_{k=1}^{K_c} w_{c,k} \mathcal{N}(x_t; \mu_{c,k}, \Sigma_{c,k}), \quad P(C_t = c|x_t) \propto P(C_t = c) f_{X_t|C_t}(x_t|c)$$

The decision rule is the usual generative one — binary log-likelihood ratio against a threshold from the log-odds, or $\arg \max_c [\log f_{X|C}(x_t|c) + \log P(C =$

c)] for multiclass — but the class-conditional density is now a **mixture**, so the decision boundary can be an **arbitrary non-linear** surface whose flexibility grows with K_c (a single-component GMM recovers the MVG/quadratic boundary).

Open-set classification. GMMs help model the heterogeneous “**none-of-the-others**” class: known classes may stay MVG, while a GMM trained on a large pool of unlabeled samples discovers homogeneous sub-clusters of the rejection class. Because different numbers of parameters are used, the resulting class-conditional likelihoods are often **not calibrated** and may need further score processing.

Variants.

- **Diagonal covariance:** fewer parameters (less overfitting/cost), possibly needing more components. This is **not** the Naive Bayes assumption: Naive Bayes would train a separate GMM for each independent subset of features.
- **Tied GMM:** all components of a single class’s GMM share one Σ . This differs from the **Tied MVG**, where the covariance is shared *across classes*.
- **Initialization** is crucial (EM finds only local optima). **K-means** can initialize the components; the **LBG** algorithm splits each component $\mu_c^\pm = \mu_c \pm \varepsilon$ (a good ε is a displacement along the principal eigenvector of Σ_c) to grow a G -component GMM into $2G$ components, running EM after each split.

Limitations. The likelihood is **unbounded** with ≥ 2 components, so EM can reach **degenerate models** ($\Sigma_c \rightarrow 0$) causing numerical problems — controlled with heuristics or constrained covariances. EM only reaches **local optima**, hence sensitivity to initialization and the use of multiple restarts. The **number of components** cannot be chosen by maximizing the likelihood (which always grows with K); **cross-validation** is used instead. Finally, reliable density estimation requires a sufficient amount of data.

13 Gaussian Mixture Models vs Generative Gaussian Models

The **Generative Gaussian Models** (MVG) and **Gaussian Mixture Models** (GMM) belong to the *same* family — both are **generative, probabilistic** models of the class-conditional density $f_{X|C}(x|c)$ used through **Bayes** — and in fact the MVG is a **special case** of the GMM. They differ in flexibility, in how the parameters are estimated, and in whether the data are labelled.

One is a Special Case of the Other

A single multivariate Gaussian is exactly a **one-component GMM** ($K = 1 \Rightarrow w_1 = 1, f_X = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$); a GMM is a **mixture** of such Gaussians, $f_{X|C}(x|c) = \sum_k w_{c,k} \mathcal{N}(x; \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k})$. Conversely, the marginal $f_X(x) = \sum_c \pi_c \mathcal{N}(x; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$ that an MVG *classifier* forms over its classes is itself a GMM. The **inference and decision rule are identical**: binary log-likelihood ratio against a log-odds threshold, multiclass $\arg \max_c [\log f_{X|C}(x|c) + \log P(C = c)]$ — only the *form* of $f_{X|C}$ changes. Because of this, the MVG produces a **linear** (tied) or **quadratic** (full/QDA) boundary, whereas the GMM produces an **arbitrary non-linear** boundary whose flexibility grows with the number of components K_c .

Closed-form ML versus EM

The MVG has **no latent variable**: labels are observed, and the parameters follow in **closed form** (per-class sample mean and covariance), a single well-posed global optimum. The GMM treats **cluster membership as a latent variable** and is trained by the iterative **EM** algorithm — E-step responsibilities $\gamma_{c,i}$, M-step weighted updates — which only reaches a **local optimum** and is sensitive to initialization (K-means / LBG). The two estimators are directly related: the GMM M-step updates $\boldsymbol{\mu}_c^* = \frac{\sum_i \gamma_{c,i} \mathbf{x}_i}{\sum_i \gamma_{c,i}}$, $\boldsymbol{\Sigma}_c^*$, w_c^* are the **soft, responsibility-weighted** version of the MVG closed-form ML, and reduce to it when responsibilities are hard 0/1. Crucially, the GMM likelihood is **ill-posed** (unbounded with ≥ 2 components, degenerate $\boldsymbol{\Sigma}_c \rightarrow 0$), whereas the MVG likelihood is bounded and well-behaved.

Supervision, Variants and Model Selection

The MVG ML estimate is **supervised** (it needs labels); the core GMM derivation is **unsupervised** ($\mathcal{D}$ treated as unlabelled), so GMMs also serve for **clustering** and **open-set** modelling of a heterogeneous “none-of-the-others” class, not only closed-set classification. Both reuse the **full / diagonal / tied** covariance vocabulary, but with different meaning: in the MVG *tied* shares Σ **across classes** and *diagonal* is Naive Bayes, while in a GMM *tied* shares Σ **across the components of one class** and *diagonal* is **not** Naive Bayes. Finally the GMM adds a **model-selection** problem — the number of components — that cannot be settled by likelihood (which always increases with K) and requires **cross-validation**; the MVG has no such choice.

Aspect	Generative Gaussian (MVG)	Gaussian Mixture Model (GMM)
Objective	ML of one Gaussian per class	ML of a mixture density (latent clusters)
Class-conditional	$\mathcal{N}(x; \mu_c, \Sigma_c)$	$\sum_k w_{c,k} \mathcal{N}(x; \mu_{c,k}, \Sigma_{c,k})$
Assumptions	class data is Gaussian	weighted mix of Gaussians; cluster = latent
Training	closed-form ML, global optimum	EM (E/M steps), local optimum, init-dependent
Likelihood	bounded, well-posed	unbounded / ill-posed, degeneracy risk
Decision rule	LLR / arg max posterior; linear (tied) or quadratic (full)	LLR / arg max posterior; arbitrary non-linear ($\uparrow K_c$)
Supervision	supervised (labels)	unsupervised core; clustering / open-set
Model selection	none ($K = 1$)	choose #components via cross-validation